

BÁO CÁO KỸ THUẬT & KIẾN TRÚC THÔNG ĐIỆP KAFKA

BÀI TẬP 4: XỬ LÝ LỖI KAFKA CONSUMER VỚI RETRY VÀ DEAD LETTER QUEUE (DLQ)

Giải cứu Consumer khỏi tình trạng kẹt cứng Poison Pill, bảo toàn luồng xử lý kho hàng StoreX

Khóa học: Microservices System Design (Rikkei Education)

Hệ thống: Nền tảng E-Commerce StoreX

Module: inventory-service lắng nghe topic "order-events"

Công nghệ: Apache Kafka, Spring Kafka 3.x, DefaultErrorHandler, DeadLetterPublishingRecoverer

1. Bối cảnh Nghiệp vụ & Hiện trạng Sự cố

Tại kho vận trung tâm **StoreX**, **InventoryConsumer** lắng nghe các sự kiện đặt hàng từ topic **order-events** (Consumer Group: **inventory-group**) để trừ số lượng sản phẩm tồn kho.

Thình thoảng, dịch vụ đặt hàng do sự cố sinh chuỗi JSON hỏng định dạng (thiếu dấu đóng ngoặc **}**). Khi Consumer đọc trúng bản tin này, việc deserialize hoặc xử lý thất bại và ném Exception. Lúc này, Consumer rơi vào trạng thái **"kẹt cứng" (Infinite Retry Loop)**: liên tục đọc lại đúng bản tin lỗi đó. Toàn bộ các đơn hàng hợp lệ phía sau bị phong tỏa, kho hàng StoreX tê liệt hoàn toàn.

```
// InventoryConsumer.java -- HIỆN TRẠNG GÂY KẸT CONSUMER
@Service
public class InventoryConsumer {
    @KafkaListener(topics = "order-events", groupId = "inventory-group")
    public void consume(OrderEvent event) {
        // Nếu event bị lỗi JSON, exception throw ra ngoài làm Consumer bị kẹt vĩnh viễn!
        inventoryService.deductStock(event.getProductId(), event.getQuantity());
    }
}
```

2. Phần 1 - Phân Tích Cơ Chế Kỹ Thuật Chuyên Sâu

2.1. Cơ chế đọc và quản lý Offset trong Apache Kafka

- Current Offset (Read Position - Vị trí đọc hiện tại):** Là con trỏ trỏ tới vị trí của bản tin tiếp theo mà Consumer chuẩn bị đọc trong một Partition. Con trỏ này tăng dần trong RAM của Consumer sau mỗi lần gọi hàm **poll()**.
- Committed Offset (Vị trí đã xác nhận thành công):** Là vị trí Offset cao nhất mà Consumer Group đã xác nhận với Kafka Broker rằng: *"Đã xử lý xong các bản tin trước offset này"*. Giá trị này được lưu bền vững vào topic hệ thống **__consumer_offsets**. Khi một Consumer bị crash hoặc xảy ra quá trình tái cân bằng (Rebalance), Consumer mới được cấp quyền sẽ căn cứ vào **Committed Offset** để tiếp tục đọc từ **Committed Offset + 1**, tránh mất mát dữ liệu.

2.2. Lý do Consumer bị kẹt khi gặp Exception mà không tự chuyển sang bản tin tiếp theo

Bản tin lỗi được gọi là "Poison Pill" (Viên thuốc độc)!

- Offset không được Commit:** Theo cơ chế At-least-once Delivery, khi phương thức **@KafkaListener** quăng một ngoại lệ chưa được bắt (Unhandled Exception), Spring Kafka hiểu rằng việc xử lý chưa hoàn tất. Vì vậy, offset của bản tin đó **KHÔNG** được commit.

2. **Cơ chế Seek ngược lại vị trí lỗi:** Mặc định, Kafka ErrorHandler sẽ gọi lệnh `consumer.seek(partition, offset)` để tua con trỏ đọc quay ngược lại vị trí bản tin vừa gặp sự cố.
3. **Vòng lặp vô hạn (Infinite Loop):** Trong lần `poll()` kế tiếp, Consumer lại lấy về đúng bản tin hỏng đó. Bản tin tiếp tục ném lỗi, tiếp tục không được commit, lại bị seek ngược lại... Quá trình này lặp lại liên tục, Consumer bị khóa cứng và không bao giờ đọc được các đơn hàng hợp lệ phía sau.

3. Phần 2 - Thiết Kế Giải Pháp & Mã Nguồn Chuẩn Hóa

3.1. Thiết kế cơ chế Retry 3 lần & Dead Letter Queue (DLQ)

- **Thử lại (Retry) tối đa 3 lần:** Sử dụng `DefaultErrorHandler` với `FixedBackOff(1000L, 2L)` (1 lần chạy ban đầu + 2 lần thử lại, mỗi lần cách nhau 1.000ms = 1s).
- **Dead Letter Queue:** Nếu sau 3 lần vẫn thất bại, `DeadLetterPublishingRecoverer` sẽ tự động trích xuất bản tin lỗi và xuất bản sang topic chết `order-events.DLT`.
- **Commit Offset & Giải phóng hàng đợi:** Sau khi đã gửi bản tin vào DLT an toàn, ErrorHandler tiến hành Commit Offset của bản tin đó. Consumer lập tức chuyển sang đọc bản tin tiếp theo, bảo đảm luồng xử lý kho hàng luôn thông suốt.
- **ErrorHandlingDeserializer:** Bọc ngoài `JsonDeserializer` để bắt các bản tin JSON hỏng cú pháp ngay từ tầng giải mã mà không làm gãy consumer loop.

3.2. Cấu hình `KafkaConsumerConfig.java`

```
@Configuration
@Slf4j
public class KafkaConsumerConfig {

    public static final String DLT_TOPIC = "order-events.DLT";

    @Bean
    public DeadLetterPublishingRecoverer deadLetterPublishingRecoverer(KafkaOperations<Object, Object> kafkaOperations) {
        return new DeadLetterPublishingRecoverer(kafkaOperations,
            (record, exception) -> {
                log.error("DeadLetterPublishingRecoverer: Bản tin tại offset [{}] đã thử đủ 3 lần. Đẩy vào DLT [{}]",
                    record.offset(), DLT_TOPIC, exception.getMessage());
                return new TopicPartition(DLT_TOPIC, record.partition());
            });
    }

    @Bean
    public DefaultErrorHandler defaultErrorHandler(DeadLetterPublishingRecoverer recoverer) {
        FixedBackOff backOff = new FixedBackOff(1000L, 2L); // 1 lần gốc + 2 lần retry = 3 lần thử
        DefaultErrorHandler errorHandler = new DefaultErrorHandler(recoverer, backOff);

        errorHandler.setRetryListeners((record, ex, deliveryAttempt) -> {
            log.warn("Retry Listener: Đang thử lại lần {} / 3 cho bản tin offset [{}]. Lỗi: {}",
                deliveryAttempt, record.offset(), ex.getMessage());
        });
        return errorHandler;
    }

    @Bean
    public ConcurrentKafkaListenerContainerFactory<Object, Object> kafkaListenerContainerFactory(
        ConsumerFactory<Object, Object> consumerFactory,
        DefaultErrorHandler defaultErrorHandler) {

        ConcurrentKafkaListenerContainerFactory<Object, Object> factory = new ConcurrentKafkaListenerContainerFactory();
        factory.setConsumerFactory(consumerFactory);
        factory.setCommonErrorHandler(defaultErrorHandler);
        return factory;
    }
}
```

3.3. Lớp Consumer Đã Sửa Đổi (`InventoryConsumer.java`)

```
@Service
@Slf4j
@RequiredArgsConstructor
public class InventoryConsumer {

    private final InventoryService inventoryService;

    @KafkaListener(topics = "order-events", groupId = "inventory-group")
    public void consume(OrderEvent event) {
        log.info("Tiếp nhận OrderEvent: orderId={}, productId={}, quantity={}",
            event.getOrderId(), event.getProductId(), event.getQuantity());

        inventoryService.deductStock(event.getProductId(), event.getQuantity());
        log.info("Khấu trừ kho thành công cho đơn hàng: {}", event.getOrderId());
    }
}
```

3.4. Lớp Giám Sát Topic Hộp Thư Chết (`InventoryDltConsumer.java`)

```
@Component
@Slf4j
public class InventoryDltConsumer {

    @KafkaListener(topics = "order-events.DLT", groupId = "inventory-dlt-group")
    public void consumeDlt(
        ConsumerRecord<String, Object> record,
        @Header(value = KafkaHeaders.DLT_ORIGINAL_OFFSET, required = false) byte[] originalOffset,
        @Header(value = KafkaHeaders.DLT_EXCEPTION_MESSAGE, required = false) String exceptionMessage) {

        log.error(" 🚨 [DLT MONITOR] Phát hiện bản tin lỗi được đưa vào Dead Letter Queue!");
        log.error("Chi tiết: Topic={}, Partition={}, Offset={}, Key={},",
            record.topic(), record.partition(), record.offset(), record.key());
        log.error("Nội dung Payload hỏng: {}", record.value());
        log.error("Nguyên nhân Exception: {}", exceptionMessage);
    }
}
```

4. Kết Quả Kiểm Thử Đơn Vị (Unit Test)

Kiểm thử hành vi Retry và định tuyến Dead Letter Queue trong `InventoryConsumerTest.java` :

Kịch bản Kiểm thử	Mục tiêu & Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
<code>testConsume_Success</code>	Gửi sự kiện đặt hàng hợp lệ (orderId: 5001, productId: 101, quantity: 3).	Hàm <code>deductStock()</code> được gọi chính xác 1 lần, không nảy sinh lỗi.	PASSED ✓
<code>testConsume_ServiceThrowsException_TriggersRetry</code>	Mô phỏng <code>inventoryService</code> ném lỗi (Database deadlock hoặc Out of stock).	Consumer ném exception ra ngoài để kích hoạt quy trình Retry của ErrorHandler.	PASSED ✓
<code>testDeadLetterPublishingRecoverer_RoutesToDlt</code>	Mô phỏng bản tin JSON hỏng, hết số lượt retry.	<code>DeadLetterPublishingRecoverer</code> gọi <code>kafkaOperations.send()</code> gửi sang <code>order-events.DLT</code> .	PASSED ✓
<code>testDefaultErrorHandler_RetryPolicyCount</code>	Kiểm tra thông số cấu hình BackOff của ErrorHandler.	Xác nhận chính xác 2 lần retry (tổng 3 lần thử), chu kỳ backoff 1.000ms.	PASSED ✓

Xác nhận kết quả: 100% các bài test đều vượt qua (Build SUCCESS). Consumer không bao giờ bị kẹt và mọi bản tin hỏng đều được cách ly sang Dead Letter Topic an toàn.

5. Kết Luận

Việc triển khai `DefaultErrorHandler` kết hợp `DeadLetterPublishingRecoverer` và `ErrorHandlingDeserializer` đã khắc phục triệt để sự cố "Consumer Ngắt Xỉu":

- Tự động phục hồi trước các lỗi mạng tạm thời nhờ cơ chế Retry 3 lần.
- Cô lập các bản tin hỏng (Poison Pills) vào Dead Letter Queue để phân tích và can thiệp sau, bảo vệ 100% luồng xử lý đơn hàng phía sau không bị gián đoạn.